

Data-Driven Estimation of Conditional Expectations, Application to Optimal Stopping and Reinforcement Learning

George V. Moustakides, Emeritus Professor
 Department of Electrical and Computer Engineering
 University of Patras, Rion, GREECE
 moustaki@upatras.gr

Abstract—When the underlying conditional density is known, conditional expectations can be computed analytically or numerically. When, however, such knowledge is not available and instead we are given a collection of training data, the goal of this work is to propose simple and purely data-driven means for estimating directly the desired conditional expectation. Because conditional expectations appear in the description of a number of stochastic optimization problems with the corresponding optimal solution satisfying a system of nonlinear equations, we extend our data-driven method to cover such cases as well. We test our methodology by applying it to Optimal Stopping and Optimal Action Policy in Reinforcement Learning.

Index Terms—Data-driven estimation, Neural networks, Optimal Stopping, Reinforcement Learning.

I. INTRODUCTION

Conditional expectations appear in a multitude of stochastic optimization problems as part of defining the corresponding optimal solutions. Characteristic examples constitute Optimal Stopping, Reinforcement Learning, Optimal Control etc. In all these cases the identification of the desired solution requires exact knowledge of the underlying data probability density which in most practical applications is not available. Nowadays with the existence of great volumes of data one may wonder whether it is possible to develop data-driven methods to solve such problems based on ideas borrowed from Machine Learning. This is exactly the goal of our present work. In particular, we intend to offer computational means for identifying the desired solution by first introducing a method capable of estimating conditional expectations of known functions. Our technique will not be based on some form of initial probability density estimation but will directly identify the conditional expectation of interest.

Since our intention is to employ Machine Learning methods, we recall that these techniques mostly employ neural networks which are trained with available data. The “design” of networks is achieved through the solution of well-defined optimization problems involving *expectations*. With the help of the Law of Large Numbers, expectations are being replaced by the existing data thus giving rise to data-driven techniques. For this reason in the sequel, our main effort focuses in proposing such optimization problems that are suitable for training and demonstrating that they indeed lead to the estimation of the functions of interest.

Before proceeding with the technical part we would like to point out that the methodology we are going to introduce is an extension of results presented in [1] and addressing the problem of likelihood ratio function estimation. This problem will also be mentioned here under the light of the richer experience we have accumulated.

II. A GENERAL OPTIMIZATION PROBLEM

Let us begin our presentation by introducing a simple optimization problem which will serve as the basis for the final, data-driven counterpart. Suppose that $\mathcal{X}$ is a random vector and consider three scalar functions $a(X), b(X), u(X)$ with $a(X) > 0$ that depend on the vector X and two additional scalar functions $\phi(z), \psi(z)$ that depend on the scalar z . Fix $a(X), b(X), \phi(z), \psi(z)$ and for each $u(X)$ define the following *average cost*

$$J(u) = E_{\mathcal{X}}[a(X)\phi(u(X)) + b(X)\psi(u(X))], \quad (1)$$

where $E_{\mathcal{X}}[\cdot]$ denotes expectation with respect to $\mathcal{X}$. We are interested in identifying the function $u_o(X)$ that solves the following optimization problem

$$\min_{u(X)} J(u) = \min_{u(X)} E_{\mathcal{X}}[a(X)\phi(u(X)) + b(X)\psi(u(X))]. \quad (2)$$

Solving (2) can in general be challenging but if we limit ourselves to a particular class of functions $\phi(z), \psi(z)$ it is possible to come up with an explicit and straightforward answer. The theorem that follows specifies this class and the corresponding optimal solution.

Theorem 1. For real z let $\omega(z)$ be a strictly increasing scalar function and denote with $\mathcal{I}$ its range of values. Select a second function $\rho(z)$ which is strictly negative and define two additional scalar functions $\phi(z), \psi(z)$ through their derivatives

$$\psi'(z) = \rho(z), \quad \phi'(z) = -\omega(z)\rho(z). \quad (3)$$

If $\text{range}(\frac{b(X)}{a(X)}) \subseteq \mathcal{I}$ then the optimal solution $u_o(X)$ of the optimization problem defined in (2) is unique and satisfies

$$\omega(u_o(X)) = \frac{b(X)}{a(X)}. \quad (4)$$

Proof: Since the function $u(X)$ which we would like to optimize depends on X and also the cost is a result of

averaging over $\mathcal{X}$, it turns out that the minimization can be performed point-wise by interchanging expectation and optimization

$$\begin{aligned} \min_{u(X)} \mathbb{E}_{\mathcal{X}} [a(X)\phi(u(X)) + b(X)\psi(u(X))] \\ = \mathbb{E}_{\mathcal{X}} \left[a(X) \min_{u(X)} \left\{ \phi(u(X)) + r(X)\psi(u(X)) \right\} \right], \end{aligned}$$

where for simplicity we denote $r(X) = \frac{b(X)}{a(X)}$. The above equality is true because by assumption $a(X) > 0$. We must emphasize that in general changing the order of expectation and minimization results in an inequality. However here because the minimization is with respect to the function $u(X)$ that depends only on the quantity that is averaged, it is straightforward to show that we actually enjoy exact equality. The minimization

$$\min_{u(X)} \left\{ \phi(u(X)) + r(X)\psi(u(X)) \right\}$$

can now be performed for *each individual* X (point-wise). Fixing X means that $u(X)$ becomes a scalar quantity u while the ratio $r(X)$ becomes a scalar number r . Consequently for each $r \in \mathcal{I}$ we need to perform a minimization with respect to u of the form

$$\min_u \{ \phi(u) + r\psi(u) \}.$$

Taking the derivative of $\phi(u) + r\psi(u)$ with respect to u and using the definition of the two functions from (3) yields

$$\phi'(u) + r\psi'(u) = (r - \omega(u))\rho(u).$$

Because of the strict increase of $\omega(\cdot)$ and the strict negativity of $\rho(\cdot)$ we conclude that the derivative is negative for u satisfying $\omega(u) < r$ and positive for $\omega(u) > r$ which implies that for $u = u_o$ satisfying $\omega(u_o) = r$ we have a unique global minimum. Since this is true for every X we deduce that the optimal function $u_o(X)$ is such that $\omega(u_o(X)) = r(X) = \frac{b(X)}{a(X)}$. This concludes the proof. ■

Because $\omega(z)$ is strictly increasing the set $\mathcal{I}$ can be either the whole real line $\mathbb{R}$ or a semi-infinite interval (a, ∞) , $(-\infty, a)$ or a finite interval (a, b) . Instead of open we can have closed intervals as well.

A. Versions of the Optimization Problem

Selecting various pairs of functions $a(X), b(X)$ in combination with the probability density of the random vector $\mathcal{X}$ produces interesting and practically meaningful optimization problems. First we address the problem of likelihood ratio identification of two densities, which is considered in detail in [1].

1) *Identification of Likelihood Ratios:* Let $g(X), f(X)$ be two possible densities for the random vector $\mathcal{X}$ and assume that $f(X) = 0$ when $g(X) = 0$ (to avoid unbounded ratios). For any scalar function $u(X)$ define the cost

$$J(u) = \mathbb{E}_{\mathcal{X}}^g [\phi(u(X))] + \mathbb{E}_{\mathcal{X}}^f [\psi(u(X))] \quad (5)$$

where $\mathbb{E}_{\mathcal{X}}^g[\cdot], \mathbb{E}_{\mathcal{X}}^f[\cdot]$ denote expectation with respect to $\mathcal{X}$ under the densities $g(X), f(X)$ respectively. Applying a simple

change of measure and denoting with $L(X) = \frac{f(X)}{g(X)}$ the likelihood ratio of the two densities we have

$$J(u) = \mathbb{E}_{\mathcal{X}}^g [\phi(u(X)) + L(X)\psi(u(X))] \quad (6)$$

which is under the form of (1). This suggests that for the minimization of (5) we can write

$$\begin{aligned} \min_{u(X)} J(u) &= \min_{u(X)} \left\{ \mathbb{E}_{\mathcal{X}}^g [\phi(u(X))] + \mathbb{E}_{\mathcal{X}}^f [\psi(u(X))] \right\} \\ &= \min_{u(X)} \mathbb{E}_{\mathcal{X}}^g [\phi(u(X)) + L(X)\psi(u(X))]. \end{aligned}$$

Application of Theorem 1 produces as optimal solution the function $u_o(X)$ that satisfies $\omega(u_o(X)) = L(X)$. Consequently we identify the likelihood ratio function of the two densities through an optimization involving expectations.

2) *Identification of Ratio of Conditional Densities:* We can extend the previous result to cover the likelihood ratio of *conditional densities*. Specifically let the pair of random vectors $(\mathcal{Y}, \mathcal{X})$ be described by two possible joint densities $g(Y, X), f(Y, X)$. Write $g(Y, X) = g(Y|X)g(X), f(Y, X) = f(Y|X)f(X)$ and as above denote $L(X) = \frac{f(X)}{g(X)}$ the likelihood ratio of the two marginals. For a scalar function $u(Y, X)$ consider the following cost

$$J(u) = \mathbb{E}_{\mathcal{Y}, \mathcal{X}}^g [L(X)\phi(u(\mathcal{Y}, \mathcal{X}))] + \mathbb{E}_{\mathcal{Y}, \mathcal{X}}^f [\psi(u(\mathcal{Y}, \mathcal{X}))]. \quad (7)$$

Applying again a change of measure we can write

$$J(u) = \mathbb{E}_{\mathcal{Y}, \mathcal{X}}^g \left[L(X)\phi(u(\mathcal{Y}, \mathcal{X})) + \frac{f(\mathcal{Y}, \mathcal{X})}{g(\mathcal{Y}, \mathcal{X})}\psi(u(\mathcal{Y}, \mathcal{X})) \right],$$

which, according to Theorem 1 when minimized over $u(Y, X)$ will yield an optimal solution of the form

$$\omega(u_o(Y, X)) = \frac{f(Y, X)}{g(Y, X)} \frac{1}{L(X)} = \frac{f(Y|X)}{g(Y|X)}.$$

Of course $L(X)$ as we have seen in the previous case can be obtained by optimizing a cost of the form of (5) which in turn can be used in the optimization of (7) to identify the likelihood ratio of the conditional densities.

3) *Identification of Conditional Expectations:* Let us now address the main problem of interest. We must point out that the advantage of the method we are going to introduce is that we can estimate *conditional expectations* by solving optimization problems involving *regular expectations*. This is particularly useful from a practical point of view since as we will see in the next section, it is straightforward to solve such problems under a data-driven setup.

Consider two scalar functions $c(Y)$ and $d(Y)$ with $c(Y) > 0$. For the pair of random vectors $(\mathcal{Y}, \mathcal{X})$ define the conditional expectations $a(X) = \mathbb{E}_{\mathcal{Y}}[c(\mathcal{Y})|\mathcal{X} = X], b(X) = \mathbb{E}_{\mathcal{Y}}[d(\mathcal{Y})|\mathcal{X} = X]$. We clearly have $a(X) > 0$. For a scalar function $u(X)$ we now define the cost

$$J(u) = \mathbb{E}_{\mathcal{Y}, \mathcal{X}} [c(\mathcal{Y})\phi(u(X)) + d(\mathcal{Y})\psi(u(X))], \quad (8)$$

where expectation is with respect to *the pair* $(\mathcal{Y}, \mathcal{X})$. Note that even though the expectation is with respect to $(\mathcal{Y}, \mathcal{X})$ the

function we would like to optimize depends *only* on X . Using the tower property of expectation the cost can be rewritten as

$$\begin{aligned} J(u) &= \mathbb{E}_X \left[\mathbb{E}_Y [c(Y)|X] \phi(u(X)) + \mathbb{E}_Y [d(Y)|X] \psi(u(X)) \right] \\ &= \mathbb{E}_X \left[a(X) \phi(u(X)) + b(X) \psi(u(X)) \right]. \end{aligned}$$

Consequently, minimizing $J(u)$ defined in (8) over $u(X)$, by application of Theorem 1 yields

$$\omega(u_o(X)) = \frac{b(X)}{a(X)} = \frac{\mathbb{E}_Y [d(Y)|X = X]}{\mathbb{E}_Y [c(Y)|X = X]}.$$

In other words it computes the ratio of the two conditional expectations. If we select $c(Y) = 1$ then

$$\omega(u_o(X)) = \mathbb{E}_Y [d(Y)|X = X],$$

namely we directly identify the conditional expectation of $d(Y)$ with respect to Y given $X = X$ as the solution of an optimization problem involving *regular* instead of conditional expectations.

This result can be further extended by considering two different densities $g(Y, X)$, $f(Y, X)$ and for a function $u(X)$ define the cost

$$J(u) = \mathbb{E}_{Y,X}^g [c(Y) \phi(u(X))] + \mathbb{E}_Y^f [d(Y) \psi(u(X))]. \quad (9)$$

In order to specify the optimizer of $J(u)$ we observe that

$$\begin{aligned} J(u) &= \mathbb{E}_X^g \left[\mathbb{E}_Y^g [c(Y)|X] \phi(u(X)) \right] + \mathbb{E}_X^f \left[\mathbb{E}_Y^f [d(Y)|X] \psi(u(X)) \right] \\ &= \mathbb{E}_X^g \left[\mathbb{E}_Y^g [c(Y)|X] \phi(u(X)) + L(X) \mathbb{E}_Y^f [d(Y)|X] \psi(u(X)) \right], \end{aligned}$$

where $L(X) = \frac{f(X)}{g(X)}$ is the likelihood ratio of the two marginal densities. Applying Theorem 1 we conclude that $J(u)$ is optimized by

$$\omega(u_o(X)) = L(X) \frac{\mathbb{E}_Y^f [d(Y)|X = X]}{\mathbb{E}_Y^g [c(Y)|X = X]}.$$

Note that the advantage of this approach is that we end up with a *single* function $u_o(X)$ that identifies the ratio of the two conditional expectations. An alternative idea would be to identify the two conditional expectations of the previous ratio separately through two different optimization problems which is clearly not as efficient.

B. Examples of functions $\omega(z), \rho(z), \phi(z), \psi(z)$

According to Theorem 1, a very important quantity in selecting the two functions $\omega(z), \rho(z)$ is the range of values $\mathcal{I}$ of $\omega(z)$ which must cover the range of values of the ratio $\frac{b(X)}{a(X)}$. The latter may not be exactly known but, as stated in the theorem, it is sufficient that $\mathcal{I}$ is a superset of this range. We will provide examples for three different cases of $\mathcal{I}$ namely $\mathcal{I} = \mathbb{R}$, $\mathcal{I} = [a, \infty)$ and $\mathcal{I} = (a, b)$ with $a < b$. The first covers the case where the range is completely unknown while the second and third refer to cases with partially known range. For example if the function $d(Y)$ for which we would like to compute the conditional expectation is nonnegative or between a and b then the same property holds true for its conditional expectation.

[A]. $\mathcal{I} = \mathbb{R}$

[A1]. $\omega(z) = z$ and $\rho(z) = -1$, results in

$$\phi(z) = \frac{z^2}{2}, \quad \psi(z) = -z.$$

This particular selection is the most popular in practice and it is known as the Mean Square Error (MSE) criterion.

[A2]. $\omega(z) = \sinh(z)$ and $\rho(z) = -e^{-0.5|z|}$ results in

$$\begin{aligned} \phi(z) &= (e^{0.5|z|} - 1) + \frac{1}{3}(e^{-1.5|z|} - 1), \\ \psi(z) &= 2\text{sign}(z)(e^{-0.5|z|} - 1). \end{aligned}$$

[A3]. $\omega(z) = \text{sign}(z)(e^{|z|} - 1)$ and $\rho(z) = -e^{-0.5|z|}$, results in

$$\phi(z) = 4 \cosh(0.5z), \quad \psi(z) = 2\text{sign}(z)(e^{-0.5|z|} - 1).$$

[B]. $\mathcal{I} = (a, \infty)$

[B1]. $\omega(z) = a + e^z$ and $\rho(z) = -\frac{1}{1+e^z}$, results in

$$\phi(z) = a \log(1 + e^{-z}) + \log(1 + e^z), \quad \psi(z) = \log(1 + e^{-z}).$$

This version when $a = 0$ resembles the *cross entropy* method introduced in [2] for the design of GANs where they employ $\phi(z) = -\log(\frac{1}{z})$, $\psi(z) = -\log(\frac{1}{1-z})$ with $z \in (0, 1)$. We can see that we propose the same functions but with z replaced in our approach by $\frac{1}{1+e^z}$ and $z \in \mathbb{R}$.

[B2]. $\omega(z) = a + e^z$ and $\rho(z) = -e^{-0.5z}$, results in

$$\phi(z) = -2ae^{-0.5z} + 2e^{0.5z}, \quad \psi(z) = 2e^{-0.5z}.$$

Remark 1. We note that when [B1] or [B2] is applied in the problem of likelihood ratio identification with $a = 0$ then

$$\omega(u_o(X)) = e^{u_o(X)} = \frac{f(X)}{g(X)} \Rightarrow u_o(X) = \log \left(\frac{f(X)}{g(X)} \right),$$

suggesting that the optimal function $u_o(X)$ is equal to the *log-likelihood ratio* of the two densities which, in problems as hypothesis testing, is often more convenient to use than the likelihood ratio itself.

In case the range is $\mathcal{I} = (-\infty, a)$ we simply consider the conditional expectation of the function $-d(Y)$ which will have a range of the form $\mathcal{I} = (-a, \infty)$.

[C]. $\mathcal{I} = (a, b)$

[C1]. $\omega(z) = a \frac{1}{1+e^z} + b \frac{e^z}{1+e^z}$ and $\rho(z) = -\frac{e^z}{1+e^z}$, results in

$$\phi(z) = \frac{b-a}{1+e^z} + b \log(1+e^z), \quad \psi(z) = -\log(1+e^z).$$

[C2]. $\omega(z) = a \frac{1}{1+e^z} + b \frac{e^z}{1+e^z}$ and $\rho(z) = -e^{-z}$, results in

$$\phi(z) = (b-a) \log \left(\frac{e^z}{1+e^z} \right) - ae^{-z}, \quad \psi(z) = e^{-z}.$$

Function $\omega(z)$ reduces to the classical sigmoid for the $(0,1)$ interval.

Remark 2. It is straightforward to propose alternative combinations of $\omega(z), \rho(z)$ that satisfy the requirements of Theorem 1. We also need to point out that for any range $\mathcal{I}$ of interest

it is always possible to employ a pair which is designed for a wider range. For example we can apply [A1] (MSE) in the case of functions with range in an interval (a, b) instead of the suggested [C1], [C2]. In fact this is common practice in the literature.

III. DATA-DRIVEN ESTIMATION

Solving the optimization problem defined in (2) in order to identify the optimal function $u_o(X)$ requires knowledge of the underlying probability density of $\mathcal{X}$. Our goal in the analysis that follows is to relax this requirement and replace it with the existence of a number of realizations of $\mathcal{X}$. This constitutes the data-driven version of the problem.

The first classical step we adopt in the direction of a data-driven approach is to replace the unknown function $u(X)$ with a parametric family $u(X, \theta)$ involving a *finite* set of parameters θ . Of course we require this family to enjoy the *universal approximation property*, namely to have the ability to approximate arbitrarily close any sufficiently smooth function provided we select a large enough model. This property is guaranteed for neural networks (shallow or deep) according to [3], [4] but it may be enjoyed by other parametric classes as well. In the sequel we limit ourselves to neural networks but similar conclusions can be claimed for any other such class.

Let us now replace $u(X)$ with a neural network $u(X, \theta)$, then the cost function in (1) becomes

$$J(\theta) = E_{\mathcal{X}}[a(X)\phi(u(X, \theta)) + b(X)\psi(u(X, \theta))], \quad (10)$$

depending only on the parameters of the network, while the corresponding optimization in (2) is replaced by

$$\min_{\theta} J(\theta) = \min_{\theta} E_{\mathcal{X}}[a(X)\phi(u(X, \theta)) + b(X)\psi(u(X, \theta))]. \quad (11)$$

For sufficiently large neural network if θ_o is the optimizer of (11) we expect the corresponding neural network $u(X, \theta_o)$ to satisfy $u(X, \theta_o) \approx u_o(X)$ with the latter being the optimizer of the original problem in (2). In other words, with the optimal finite dimensional version of the problem defined in (11) we approximate the optimal function solving its infinite dimensional counterpart in (2). As in the original problem in (2), the finite dimensional version (11) is defined in terms of the pdf of $\mathcal{X}$. Therefore, let us now assume that we are under a data-driven setup.

As we can see the cost in (10) can be put under the general form

$$J(\theta) = E_{\mathcal{X}}[h(X, \theta)],$$

where $h(X, \theta)$ is a scalar deterministic function and (11) corresponds to solving the optimization problem

$$\min_{\theta} J(\theta) = \min_{\theta} E_{\mathcal{X}}[h(X, \theta)], \quad (12)$$

for the case where we have a set of realization $\{X_1, \dots, X_n\}$ (training set) of $\mathcal{X}$ which replaces the exact knowledge of the probability density function of $\mathcal{X}$.

The obvious possibility is to evoke the classical Law of Large Numbers (LLN) and approximate the expectation by the following data-driven cost

$$\hat{J}(\theta) = \frac{1}{n} \sum_{i=1}^n h(X_i, \theta) \approx J(\theta).$$

Cost $\hat{J}(\theta)$ is a completely known function of θ and can therefore be minimized with the help, for example, of the Gradient Descent (GD) iterative algorithm

$$\theta_t = \theta_{t-1} - \mu \sum_{i=1}^n \nabla_{\theta} h(X_i, \theta_{t-1}) \quad (13)$$

where $\mu > 0$ is the step-size (learning rate) and $\nabla_{\theta} h(X, \theta)$ is the gradient with respect to θ of the scalar function $h(X, \theta)$. Note that we have absorbed in μ the division by the constant n . As we can see, the GD in each iteration requires the computation of the gradients of all realizations which could be computationally demanding especially when the training set is large.

An alternative approach would be to employ the Stochastic Gradient Descent (SGD) algorithm which consists in applying the update

$$\theta_t = \theta_{t-1} - \mu \nabla_{\theta} h(X_t, \theta_{t-1}) \quad (14)$$

where we retain only a *single* gradient evaluated for a realization X_t from the training set. In each iteration we use a different data point and when all realization are exhausted (epoch) then we reuse them starting from the beginning of the training set (after possibly randomly shuffling the data).

There is also the version where in each iteration we employ micro-blocks of m elements from the training set and replace the single gradient of the classical version with the average of the gradients computed over the block (again division by m is absorbed in μ)

$$\theta_t = \theta_{t-1} - \mu \sum_{i=1}^m \nabla_{\theta} h(X_{(t-1)m+i}, \theta_{t-1}).$$

The SGD is clearly computationally less demanding per iteration than the GD.

Relating (14) to the solution of the original problem defined in (12) is not as straightforward as in (13) where we simply call upon the LLN. A more sophisticated Stochastic Approximation theory [10] is required to demonstrate that the SGD version indeed provides the desired estimate. Furthermore, as it has been observed in practice, the behavior of the two algorithms (GD vs SGD) with respect to convergence speed and capability to avoid undesirable local minima can be quite different.

A. Data-Driven Likelihood Ratio Estimation

In the previous section when we discussed the problem of likelihood ratio identification we considered the existence of two densities $g(X), f(X)$ and the need to minimize the performance measure in (5) with respect to the unknown function $u(X)$.

Following similar steps as the ones described above, we first replace $u(X)$ with a neural network $u(X, \theta)$ then, instead of the two densities we assume existence of two datasets $\{X_1^g, \dots, X_{n_g}^g\}$ and $\{X_1^f, \dots, X_{n_f}^f\}$ sampled from $g(X), f(X)$ respectively. Using the definitions in (3) and denoting with $\nabla_{\theta} u(X, \theta)$ the gradient with respect to θ of the neural network $u(X, \theta)$ we can write for the GD iteration

$$\begin{aligned} \theta_t &= \theta_{t-1} + \\ &\mu \left\{ \frac{1}{n_g} \sum_{i=1}^{n_g} \omega(u(X_i^g, \theta_{t-1})) \rho(u(X_i^g, \theta_{t-1})) \nabla_{\theta} u(X_i^g, \theta_{t-1}) \right. \\ &\quad \left. - \frac{1}{n_f} \sum_{j=1}^{n_f} \rho(u(X_j^f, \theta_{t-1})) \nabla_{\theta} u(X_j^f, \theta_{t-1}) \right\}. \end{aligned}$$

For the SGD version we assume that the two datasets are randomly mixed with the samples retaining their labels (i.e. whether they come from g or f). The samples are used one after the other and the update per iteration depends on the corresponding label of the sample. Specifically when X_t is from g then

$$\theta_t = \theta_{t-1} + \frac{\mu}{n_g} \omega(u(X_t, \theta_{t-1})) \rho(u(X_t, \theta_{t-1})) \nabla_{\theta} u(X_t, \theta_{t-1}),$$

whereas when X_t is from f then

$$\theta_t = \theta_{t-1} - \frac{\mu}{n_f} \rho(u(X_t, \theta_{t-1})) \nabla_{\theta} u(X_t, \theta_{t-1}).$$

When $n_g = n_f$ we can simplify the SGD algorithm by not mixing the two datasets and by employing in each iteration one sample from each dataset as follows (division by $n_f = n_g$ is absorbed in μ)

$$\begin{aligned} \theta_t &= \theta_{t-1} + \mu \left\{ \omega(u(X_t^g, \theta_{t-1})) \rho(u(X_t^g, \theta_{t-1})) \nabla_{\theta} u(X_t^g, \theta_{t-1}) \right. \\ &\quad \left. - \rho(u(X_t^f, \theta_{t-1})) \nabla_{\theta} u(X_t^f, \theta_{t-1}) \right\}. \end{aligned}$$

Gradient type algorithms for estimating the likelihood ratio of conditional probability densities can be designed in a similar way.

B. Data-Driven Estimation of Conditional Expectations

We would like to emphasize once more that the advantage of the proposed methodology is the fact that we can estimate conditional expectations by solving optimization problems involving regular expectations. Let us recall the cost in (8) and consider the simplified version with $c(Y) = 1$. For a function $d(Y)$ we are interested in estimating the conditional expectation $\mathbb{E}_Y[d(Y)|\mathcal{X} = X]$. Following our usual practice, the function $u(X)$ is replaced by a neural network $u(X, \theta)$ and the joint density $f(Y, X)$ by a collection of pairs $\{(Y_1, X_1), \dots, (Y_n, X_n)\}$ sampled from it. The finite dimensional version of the cost function in (8) then becomes

$$J(\theta) = \mathbb{E}_{Y, X} [\phi(u(X, \theta)) + d(Y) \psi(u(X, \theta))],$$

which suggests the following data-driven cost

$$\hat{J}(\theta) = \frac{1}{n} \sum_{i=1}^n \left\{ \phi(u(X_i, \theta)) + d(Y_i) \psi(u(X_i, \theta)) \right\}$$

and the corresponding GD algorithm for its minimization

$$\begin{aligned} \theta_t &= \theta_{t-1} - \mu \sum_{i=1}^n \left\{ d(Y_i) - \omega(u(X_i, \theta_{t-1})) \right\} \times \\ &\quad \rho(u(X_i, \theta_{t-1})) \nabla_{\theta} u(X_i, \theta_{t-1}). \end{aligned} \quad (15)$$

For the SGD we can write

$$\begin{aligned} \theta_t &= \theta_{t-1} - \mu \left\{ d(Y_t) - \omega(u(X_t, \theta_{t-1})) \right\} \times \\ &\quad \rho(u(X_t, \theta_{t-1})) \nabla_{\theta} u(X_t, \theta_{t-1}). \end{aligned} \quad (16)$$

When θ_t converges to θ_o , we expect that $\omega(u(X, \theta_o)) \approx \mathbb{E}_Y[d(Y)|\mathcal{X} = X]$ without any knowledge of the joint or the conditional probability density of $\mathcal{Y}$ given $\mathcal{X}$.

Remark 3. As we can see from (15), (16) for the updates we require the two initial functions $\omega(z)$, $\rho(z)$ and not $\phi(z)$, $\psi(z)$. The latter are needed only for the computation of the corresponding cost $\hat{J}(\theta_t)$ which can be used to monitor the stability and convergence of the iterations. In fact, in order to experience stable updates in (15), (16) we need to ensure that $d(Y) \in \mathcal{I}$. In other words, the range of $\omega(z)$ must cover the range of values of $d(Y)$.

We should mention that in (15), (16) it is very common to employ the ADAM version [5] where gradient elements are normalized by the square root of their running power. This idea establishes a more uniform convergence for the components of the parameter vector. Powers are estimated with the help of exponential windowing with a forgetting factor λ .

The function $\omega(z)$ which is applied after the computation of the output of the neural network $u(X, \theta_o)$ can be seen as an *output activation function*. We should however emphasize that the updates in (15) or (16) would have been different if we had considered $\omega(z)$ as part of the neural network from the start. Indeed the gradient with respect to the parameters, unlike in (15) and (16), would have also included the derivative $\omega'(z)$ which is now absent.

C. Numerical Computation of Conditional Expectations

In order to evaluate the proposed data-driven estimation method we would need to compare its results with the exact conditional expectation, for characteristic examples. Since it is not always possible to analytically compute the conditional expectation we would like to offer a computational technique based on simple numerical integration rules.

For the pair $(\mathcal{Y}, \mathcal{X})$ where $\mathcal{Y}$ and $\mathcal{X}$ are *scalars* let $f(Y|X)$ be the conditional pdf and $F(Y|X)$ the corresponding conditional cdf. The conditional expectation $\mathbb{E}_Y[d(Y)|\mathcal{X} = X]$ for a known function $d(Y)$ can then be written as

$$\begin{aligned} u(X) &= \mathbb{E}_Y[d(Y)|\mathcal{X} = X] \\ &= \int d(Y) f(Y|X) dY = \int d(Y) dF(Y|X). \end{aligned} \quad (17)$$

If we sample Y and X over sufficiently large intervals at $\{Y_1, \dots, Y_n\}$ and $\{X_1, \dots, X_m\}$ respectively then we can generate the doubly indexed sequence $F(Y_j|X_i)$ and the two sequences $d(Y_j)$, $u(X_i)$ where $j = 1, \dots, n$ and $i = 1, \dots, m$. We note that the values $u(X_i)$ are the samples of the conditional expectation we would like to determine. By averaging the forward and backward version of the rectangle rule in the second integral in (17), the conditional expectation can enjoy the following approximation

$$u(X_i) \approx \frac{1}{2} \sum_{j=1}^{n-1} d(Y_j) [F(Y_{j+1}|X_i) - F(Y_j|X_i)]$$

$$\begin{aligned}
& + \frac{1}{2} \sum_{j=2}^n d(Y_j) [F(Y_j|X_i) - F(Y_{j-1}|X_i)] \\
& = d(Y_1) \frac{F(Y_2|X_i) - F(Y_1|X_i)}{2} \\
& + d(Y_2) \frac{F(Y_3|X_i) - F(Y_2|X_i)}{2} + \dots \\
& + d(Y_{n-1}) \frac{F(Y_n|X_i) - F(Y_{n-2}|X_i)}{2} \\
& + d(Y_n) \frac{F(Y_n|X_i) - F(Y_{n-1}|X_i)}{2}.
\end{aligned}$$

We note that the first and last term in the last sum are different from the intermediate terms. The above formula can be conveniently rewritten as a matrix/vector product. Indeed if we define the vectors $\mathbf{U} = [u(X_1), \dots, u(X_m)]^\top$, $\mathbf{D} = [d(Y_1), \dots, d(Y_n)]^\top$ and the matrix $\mathcal{F}$ of dimensions $m \times n$ with the i -th row of the matrix having the following elements $(\mathcal{F})_{i1} = 0.5[F(Y_2|X_i) - F(Y_1|X_i)]$, $(\mathcal{F})_{ij} = 0.5[F(Y_{j+1}|X_i) - F(Y_{j-1}|X_i)]$, $j = 2, \dots, n-1$ and $(\mathcal{F})_{in} = 0.5[F(Y_n|X_i) - F(Y_{n-1}|X_i)]$, then we can write $\mathbf{U} \approx \mathcal{F} \mathbf{D}$, suggesting that the product $\mathcal{F} \mathbf{D}$ provides an approximation to the sampled values of the conditional expectation.

In case we have available the conditional pdf $f(Y|X)$ but it is not possible to compute analytically the conditional cdf $F(Y|X)$ we can use the classical version of the forward and backward rectangle rule applied to the first integral in (17), namely

$$\begin{aligned}
u(X_i) & \approx \frac{1}{2} \sum_{j=1}^{n-1} d(Y_j) f(Y_j|X_i) (Y_{j+1} - Y_j) \\
& + \frac{1}{2} \sum_{j=2}^n d(Y_j) f(Y_j|X_i) (Y_j - Y_{j-1}) \\
& = \sum_{j=1}^{n-1} \frac{1}{2} \left(d(Y_j) f(Y_j|X_i) + d(Y_{j+1}) f(Y_{j+1}|X_i) \right) (Y_{j+1} - Y_j),
\end{aligned}$$

leading to the trapezoidal rule. This can also be combined to the more convenient matrix/vector product by properly redefining the matrix $\mathcal{F}$.

As mentioned, the numerical technique will be used whenever it is impossible to obtain an analytic formula and will serve as a point of reference for the proposed data-driven method. We should of course keep in mind that the numerical method requires exact knowledge of the conditional pdf (cdf) whereas the data-driven technique we developed relies only on training data.

Even though the numerical method is presented for the scalar case it is possible to extend it to accommodate random vectors $\mathcal{Y}, \mathcal{X}$. However, very quickly we realize that as we consider larger dimensions the amount of necessary computations increases dramatically with the method suffering from the ‘‘curse of dimensionality’’.

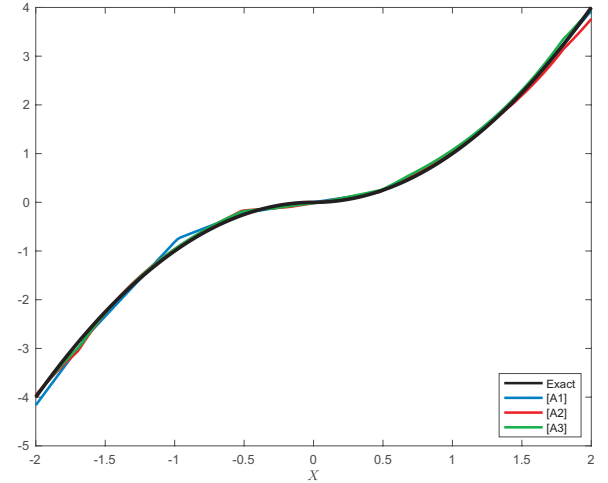
When we apply this numerical method we have to make sure that the intervals we sample are such that the conditional probability mass which is left outside the sampled Y -interval is negligible for all sampled values of X and so is the probability mass left outside the sampled X -interval. If this is not the case

then changing the size of the intervals produces inconsistent and therefore questionable results.

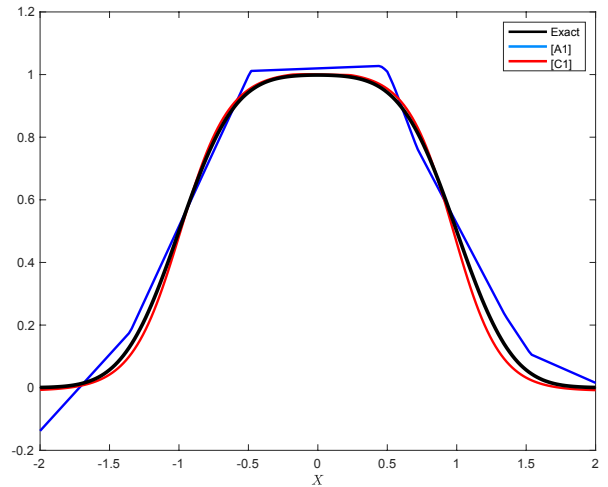
D. Examples

Let us apply our idea to two examples. Consider $\mathcal{Y}, \mathcal{X}$ scalar random variables related through the equations: a) $\mathcal{Y} = \text{sign}(\mathcal{X}) \mathcal{X}^2 + \mathcal{W}$, and b) $\mathcal{Y} = \mathbb{1}_{[-1,1]}(\mathcal{X} + \mathcal{W})$, where $\mathbb{1}_A(X)$ denotes the indicator function of the set A . In both cases $\mathcal{X}$ is standard Gaussian while $\mathcal{W}$ plays to role of noise which we assume to be zero-mean Gaussian with variance $\sigma_{\mathcal{W}}^2 = 0.1$. We are interested in estimating $E_{\mathcal{Y}}[\mathcal{Y}|\mathcal{X} = X]$ and it is straightforward to see that for a) we have $E_{\mathcal{Y}}[\mathcal{Y}|\mathcal{X} = X] = \text{sign}(X)X^2$, whereas for b) $E_{\mathcal{Y}}[\mathcal{Y}|\mathcal{X} = X] = \Phi(\frac{1-X}{\sigma_{\mathcal{W}}}) - \Phi(\frac{-1-X}{\sigma_{\mathcal{W}}})$, where $\Phi(x)$ is the cdf of the standard Gaussian. Here we have an analytic formula for both conditional expectations but even if we had used the numerical method proposed in Section III-C the two results would have been indistinguishable.

To test our data-driven method we generate $n = 200$ random pairs (Y_i, X_i) and train a shallow neural network with a single



(a)



(b)

Fig. 1. Estimation of conditional expectation $E_{\mathcal{Y}}[\mathcal{Y}|\mathcal{X} = X]$ when (a) $\mathcal{Y} = \text{sign}(\mathcal{X}) \mathcal{X}^2 + \mathcal{W}$ and (b) $\mathcal{Y} = \mathbb{1}_{[-1,1]}(\mathcal{X} + \mathcal{W})$ with $n = 200$ training samples and using a shallow network with hidden layer of size 50.

hidden layer of size 50 and ReLU activations. To the obvious question why we select this specific size we can say that this is a fundamental problem in neural networks and currently there is no analytically trustworthy answer as to which is the appropriate network size and how must be related to the size of the training dataset.

We apply the GD algorithm depicted in (15) and adopt the ADAM version (5) that normalizes each gradient element with the square root of its running power. We select a step-size equal to $\mu = 0.001$ and exponential windowing with forgetting factor $\lambda = 0.99$ for the power estimates. We run the algorithms for 2000 iterations which is sufficient for convergence as we could have verified if we had plotted the corresponding costs $\hat{J}(\theta_t)$. The parameter vectors θ_o we converge to, are used to estimate the conditional expectations as $\omega(u(X, \theta_o))$ and by sampling the range $[-2, 2]$ of X uniformly we compare with the corresponding values of the exact formulas mentioned above.

With Example a) since the range $\mathcal{I} = \mathbb{R}$, we test versions [A1], [A2], [A3] from Section II-B. As we can see from Fig. 1(a) here the classical MSE version [A1] has comparable performance with the other two alternatives and all three methods approximate sufficiently well the exact function. Let us now turn to Example b) where the range $\mathcal{I}$ of $\mathcal{Y}$ and $\omega(z)$ is $[0, 1]$. In this case we apply the classical MSE version [A1] and version [C1] with $a = -0.01, b = 1.01$ which slightly overcovers $\mathcal{I}$. As we observe in Fig. 1(b) knowledge of the range and selection of the appropriate functions $\omega(z), \rho(z)$ may improve the estimation quality, dramatically.

Remark 4. From the analytical computation of the conditional expectation it is clear that this function does not depend on the distribution of $\mathcal{X}$. Since our data pairs (Y_i, X_i) are sampled from the joint density, one may wonder how the marginal density of $\mathcal{X}$ affects our estimates. Basically it is expected that the proposed methodology will make notable estimation errors for values of $\mathcal{X}$ with small likelihood. To understand this fact, consider the extreme case where an interval of values of $\mathcal{X}$ has zero probability of occurrence, then no samples from this interval can appear in the training set. It is therefore unrealistic to expect that our estimate will be accurate for such X values. A similar conclusion applies when the likelihood of the interval is small and we obtain only very few (or even no) samples from the interval in the training set. This fact might seem as a serious weakness of our method however making errors at points that *never* occur or at points that occur very rarely might not be so crucial from a practical point of view.

IV. SYSTEM OF EQUATIONS INVOLVING CONDITIONAL EXPECTATIONS

The estimation method we introduced can be employed to compute solutions of *systems of equations* expressed with the help of conditional expectations. Such systems occur in several well-known stochastic optimization problems. Characteristic examples constitute the problem of Optimal Stopping in Markov processes and the Optimal Action Policy in Reinforcement Learning which we consider in detail after introducing our general setup.

Let $h^j(Y, u^1, \dots, u^K)$, $j = 1, \dots, K$ be K deterministic and known scalar functions with $u^1, \dots, u^K$ scalar variables. Suppose there are also K different conditional densities $f^j(Y|X)$, $j = 1, \dots, K$ with Y and X of the *same length*. Define the following system of equations which we are interested in solving for the scalar functions $\mathcal{U}^1(X), \dots, \mathcal{U}^K(X)$

$$\mathcal{U}^j(X) = \mathbb{E}_{\mathcal{Y}} [h^j(\mathcal{Y}, \mathcal{U}^1(\mathcal{Y}), \dots, \mathcal{U}^K(\mathcal{Y})) | \mathcal{X} = X], \quad (18)$$

for $j = 1, \dots, K$ and where expectation is with respect to $\mathcal{Y}$ conditioned on $\mathcal{X} = X$ using the conditional density $f^j(Y|X)$.

A. Numerical Solution

We first apply the numerical method of Section III-C to find a numerical solution when the K conditional densities are known. We select the same sufficiently large interval for Y and X which we sample at the same points $Y_i = X_i, i = 1, \dots, n$. We must assure that what is left outside the interval has very small probability conditioned on every value X_i and this must be true for all K conditional densities. Call $\mathbf{U}^j = [\mathcal{U}^j(Y_1), \dots, \mathcal{U}^j(Y_n)]^\top, j = 1, \dots, K$ the sampled version of the K solution functions which we would like to determine. For each conditional density $f^j(Y|X)$ define the matrix $\mathcal{F}^j$ as explained in Section III-C. Finally, form K vectors $\mathbf{H}^j(\mathbf{U}^1, \dots, \mathbf{U}^K), j = 1, \dots, K$ with their elements defined as follows

$$\left(\mathbf{H}^j(\mathbf{U}^1, \dots, \mathbf{U}^K) \right)_i = h^j(Y_i^j, (\mathbf{U}^1)_i, \dots, (\mathbf{U}^K)_i),$$

$i = 1, \dots, n, j = 1, \dots, K$, where $(Z)_i$ denotes the i th element of the vector Z . In other words the i th element of $\mathbf{H}^j$ is equal to the function h^j evaluated at $Y = Y_i^j$ with the variables u^j replaced by the i th elements of the vectors $\mathbf{U}^j$. With these definitions it is clear that the sampled version of the equation in (18) takes the following matrix/vector product form

$$\mathbf{U}^j = \mathcal{F}^j \times \mathbf{H}^j(\mathbf{U}^1, \dots, \mathbf{U}^K), \quad j = 1, \dots, K.$$

The previous system can be solved iteratively by iterating on the unknown vectors

$$\mathbf{U}_t^j = \mathcal{F}^j \times \mathbf{H}^j(\mathbf{U}_{t-1}^1, \dots, \mathbf{U}_{t-1}^K), \quad j = 1, \dots, K.$$

This will be the method we are going to apply to compute the numerical solution in the problems of Optimal Stopping and Reinforcement Learning when analytic formulas are impossible.

B. Data-Driven Solution

We are now considering the data-driven version and the corresponding solution of the system of equations. Key observation for solving the system in (18) is that all functions of interest are defined in terms of conditional expectations and therefore, following our main idea, each conditional expectation can be estimated using the methodology we developed in Section III-B.

Our data-driven setup is as follows: We are given K datasets $\{(Y_1^j, X_1^j), \dots, (Y_{n_j}^j, X_{n_j}^j)\}, j = 1, \dots, K$ that replace the conditional densities $f^j(Y|X), j = 1, \dots, K$. Since we are

interested in computing the functions $\mathcal{U}^j(X), j = 1, \dots, K$, we approximate each function $\mathcal{U}^j(X)$ with a neural network $u(X, \theta^j)$. Consider first the GD version for estimating the network parameters. Because of (18) we need to apply (15) with $d(Y) = h_j(Y, \mathcal{U}^1(Y), \dots, \mathcal{U}^K(Y))$, namely

$$\theta_t^j = \theta_{t-1}^j - \mu \sum_{i=1}^{n_j} \left\{ h_j(Y_i^j, \mathcal{U}^1(Y_i^j), \dots, \mathcal{U}^K(Y_i^j)) - \omega(u(X_i^j, \theta_{t-1}^j)) \right\} \rho(u(X_i^j, \theta_{t-1}^j)) \nabla_{\theta} u(X_i^j, \theta_{t-1}^j).$$

Unfortunately the previous formula is impossible to use for the parameter updates because the functions $\mathcal{U}^j(Y)$ on the right hand side are the ones we are actually attempting to estimate. We recall that our estimation method of conditional expectation results in a final approximation of the form $\omega(u(X, \theta_o^j)) \approx \mathcal{U}^j(X)$ consequently, due to (18), $\omega(u(Y, \theta_o^j))$ could replace $\mathcal{U}^j(Y)$ in the previous updates. But this is still problematic since the limits θ_o^j are not known in advance. We therefore propose at iteration t to employ the most recent estimate $\omega(u(Y, \theta_{t-1}^j))$ of $\mathcal{U}^j(Y)$. This selection clearly allows for computations and produces the following updates

$$\theta_t^j = \theta_{t-1}^j - \mu \sum_{i=1}^{n_j} \left\{ h_j(Y_i^j, \omega(u(Y_i^j, \theta_{t-1}^1)), \dots, \omega(u(Y_i^j, \theta_{t-1}^K))) - \omega(u(X_i^j, \theta_{t-1}^j)) \right\} \rho(u(X_i^j, \theta_{t-1}^j)) \nabla_{\theta} u(X_i^j, \theta_{t-1}^j), \quad (19)$$

for $j = 1, \dots, K$. We note that in the iteration for θ_t^j we use the dataset corresponding to $f^j(Y|X)$. We also observe that the updates are performed in parallel since each iteration involves the update of all K network parameter vectors $\theta_t^j, j = 1, \dots, K$. Of course it is not necessary to use the same $\omega(\cdot), \rho(\cdot)$ functions or the same neural network configuration when approximating the desired functions $\mathcal{U}^j(X)$.

In case we prefer to employ the SGD then the K datasets must be (randomly) mixed with each pair retaining its label. Then at iteration t if we select to process the pair (Y_t, X_t) with label j we apply

$$\theta_t^j = \theta_{t-1}^j - \mu \left\{ h_j(Y_t, \omega(u(Y_t, \theta_{t-1}^1)), \dots, \omega(u(Y_t, \theta_{t-1}^K))) - \omega(u(X_t, \theta_{t-1}^j)) \right\} \rho(u(X_t, \theta_{t-1}^j)) \nabla_{\theta} u(X_t, \theta_{t-1}^j)$$

$$\theta_t^\ell = \theta_{t-1}^\ell, \ell \neq j.$$

The limiting values $\theta_o^j, j = 1, \dots, K$ provide the estimates $\omega(u(X, \theta_o^j)) \approx \mathcal{U}^j(X)$ which are approximations of the solution of the system of equations.

This is the general data-driven approach and computational methodology we propose. To test its effectiveness, we apply it to Optimal Stopping and Reinforcement Learning.

C. Markov Optimal Stopping

Consider a homogeneous Markov process $\{\mathcal{X}_t\}$ which is observed (sampled) sequentially, namely at each time t we observe a new point $\mathcal{X}_t$. We can decide to stop sampling at any time T which can either be deterministic or random. When

the decision to stop at $\{T = t\}$ is based on the available information $\{X_0, \dots, X_t\}$ accumulated up to time t , then T is called a *stopping time adapted to $\{\mathcal{X}_t\}$* .

We are interested in the minimization of the following exponentially discounted average cost over a stopping time T [6], [7]

$$\mathcal{U}(X) = \inf_T \mathbb{E} \left[\alpha^T p(\mathcal{X}_T) + \sum_{t=0}^{T-1} \alpha^t q(\mathcal{X}_t) \mid X_0 = X \right],$$

where α is the exponential factor, $p(\cdot), q(\cdot)$ are known functions with $q(\mathcal{X}_t)$ expressing the cost of sampling at t and $p(\mathcal{X}_t)$ the cost of stopping at t . If we consider the infinite horizon version (no hard limit on T) then we know (see [8], Page 70) that $\mathcal{U}(X)$ satisfies the equation

$$\mathcal{U}(X) = \mathbb{E}_{X_1} [\min \{p(\mathcal{X}_1), q(\mathcal{X}_1) + \alpha \mathcal{U}(\mathcal{X}_1)\} \mid X_0 = X], \quad (20)$$

while the stopping time T_o defined as

$$T_o = \inf \{t \geq 0 : p(X_t) \leq q(X_t) + \alpha \mathcal{U}(X_t)\}, \quad (21)$$

delivers the optimal cost $\mathcal{U}(X)$ and is therefore optimum. The general system in (18) clearly covers equation (20) for Optimal Stopping by selecting $K = 1$ and $h^1(Y, u^1) = \min\{p(Y), q(Y) + \alpha u^1\}$.

A numerical solution for this problem can be devised based on the method introduced in Section IV-A when the transition density is known and X is scalar. If we select a sufficiently large interval and sample it at $\{X_1, \dots, X_n\}$ we generate the vector $U = [u(X_1), \dots, u(X_n)]^\top$ of samples from the unknown solution, the known vectors $P = [p(X_1), \dots, p(X_n)]^\top$, $Q = [q(X_1), \dots, q(X_n)]^\top$ and, finally the matrix $\mathcal{F}$ as detailed in Section III-C. Then the sampled version of (20) takes the form

$$U = \mathcal{F} \times \min\{P, Q + \alpha U\}$$

where the “min” is applied on an element-by-element basis on the two vectors P and $Q + \alpha U$. This equation can be solved by iterating over U as follows

$$U_t = \mathcal{F} \times \min\{P, Q + \alpha U_{t-1}\}, \quad U_0 = P, \quad (22)$$

and considering the limit of U_t as approximating the sampled form of the desired solution.

Example with AR(1) Process: Let $\{\mathcal{X}_t\}$ be a homogeneous AR(1) process of the form $\mathcal{X}_t = 0.9\mathcal{X}_{t-1} + \sqrt{5}\mathcal{W}_t$ where $\{\mathcal{W}_t\}$ is standard i.i.d. Gaussian noise sequence. The sampling cost is selected $q(X) = 0.1$ while the stopping cost $p(X)$ is depicted in Fig. 2(a) in gray. The exponential discount factor is set to $\alpha = 1$. Because of this selection we can prove that $\min_X p(X) \leq \mathcal{U}(X) \leq \max_X p(X)$ which can define a possible range $\mathcal{I}$ for the solution.

Since solving (20) analytically is impossible we apply the numerical method we detailed above. In particular we select the interval $[-30, 30]$ and sample it uniformly at 5000 points. We form the matrix $\mathcal{F}$ and the vectors U, P, Q and iterate over U_t for 1000 times. In Fig. 2(a), as we said, we can see the stopping cost $p(X)$ in gray and the numerical solution elevated by the sampling cost $q(X)$ in black. In other words we plot $Q + U$ because, according to (21), this sum must be compared

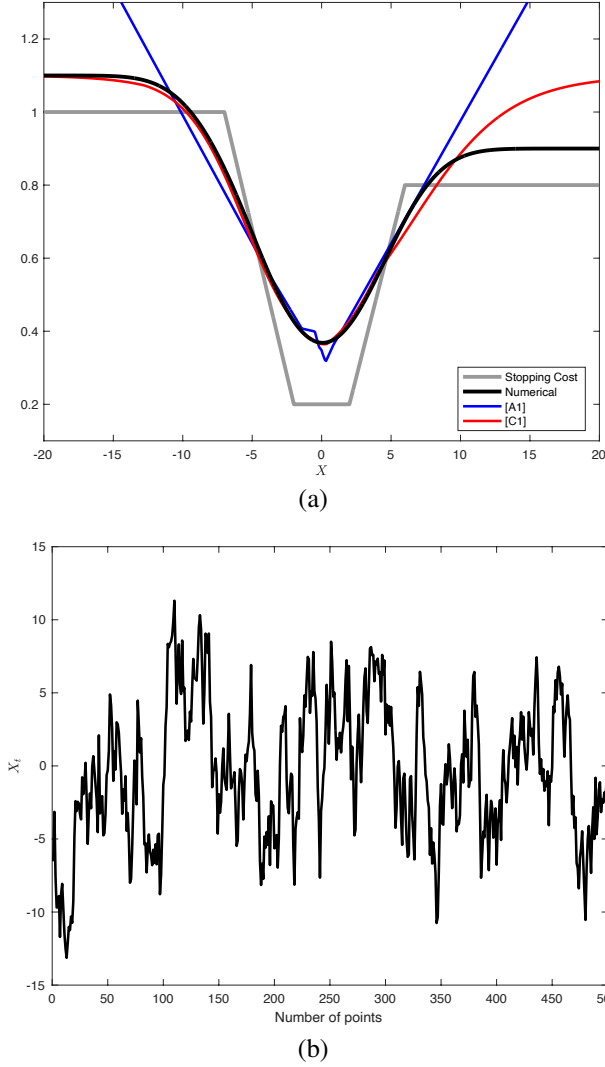


Fig. 2. (a) Numerical and Data-Driven estimation of the optimal average cost with $n = 500$ samples, using a shallow networks with hidden layer of size 100 nodes. (b) The training set with 500 consecutive realization of the Markov process $\{X_t\}$.

with the stopping cost in order to decide optimally whether to stop or continue sampling at any time t .

For our proposed data-driven estimation method, we generate 500 consecutive realizations of the AR(1) process and use them to train a shallow network with hidden layer of size 100 and ReLU activations. We apply the GD algorithm depicted in (19) with $K = 1$, step-size $\mu = 0.001$ and forgetting factor for the ADAM version equal to $\lambda = 0.99$. We run [A1] and [C1] with $\mathcal{I} = [0.2, 1]$ (the lower and upper bound of $p(X)$) for 2000 iterations. Version [A1] (MSE) is plotted with blue line whereas [C1] with red (both elevated by $q(X)$). We realize again that knowledge of the range can result in better estimates. Observing Fig. 2(a) one may argue that the error in [C1] is more pronounced when $X \geq 10$. This is because, as we can see from Fig. 2(b) where we plot the 500 consecutive realizations of the Markov process, there are very few samples in the training set with such values. According to Remark 4, this is the reason why we may experience estimates with large errors.

D. Optimal Action Policy in Reinforcement Learning

A second perhaps more popular nowadays problem is the optimal action policy in Reinforcement Learning. Suppose $\{\mathcal{S}_t\}$ is a Markov controlled process with the action taking discrete values in the set $\{1, 2, \dots, K\}$ and with each action corresponding to a different transition density. Denote with $E^j[\cdot]$ the conditional expectation with respect to the transition density of action j . With every state S assume there is a reward $\mathcal{R}(S)$ where $\mathcal{R}(\cdot)$ is a known scalar deterministic function. We are looking for the best action policy that will result in maximal average reward over an infinite and exponentially discounted time horizon of the form

$$\begin{aligned} \mathcal{Q}(S) &= \max_{j_0, j_1, \dots} E[\mathcal{R}(\mathcal{S}_1) + \gamma \mathcal{R}(\mathcal{S}_2) + \gamma^2 \mathcal{R}(\mathcal{S}_3) + \dots] | \mathcal{S}_0 = S] \\ &= \max_{j_0} E_{\mathcal{S}_1}[\mathcal{R}(\mathcal{S}_1) + \gamma \mathcal{Q}(\mathcal{S}_1) | \mathcal{S}_0 = S] \end{aligned}$$

where $0 \leq \gamma < 1$ is the geometric discount factor.

If at time $t = 0$ we observe state $\mathcal{S}_0 = S$ and we decide in favor of action j and *after this point we always use optimal action policy* then let us call the resulting reward $\mathcal{U}^j(S)$. It can then be proved (see [9], Eq. (4.2), Page 90) that these functions satisfy the following system of equations

$$\mathcal{U}^j(S) = E_{\mathcal{S}_1}^j \left[\mathcal{R}(\mathcal{S}_1) + \gamma \max_{1 \leq \ell \leq K} \mathcal{U}^\ell(\mathcal{S}_1) | \mathcal{S}_0 = S \right], \quad (23)$$

where $j = 1, \dots, K$. Also

$$\mathcal{Q}(S) = \max_{1 \leq \ell \leq K} \mathcal{U}^\ell(S). \quad (24)$$

The previous equality suggests that if we know the functions $\mathcal{U}^j(S)$, then at time t if we observe state $\mathcal{S}_t = S_t$ the action j_t° which guarantees optimal reward is

$$j_t^\circ = \arg \max_{1 \leq \ell \leq K} \mathcal{U}^\ell(S_t).$$

The system of equations (23) is a special case of the general system in (18). Indeed, we need to select all functions to be the same, that is, $h^j(\cdot) = h(\cdot)$ with $h(Y, u^1, \dots, u^K) = \mathcal{R}(Y) + \gamma \max_{1 \leq \ell \leq K} u^\ell$.

It is only for simplicity we have considered a finite number of actions. Applying similar analysis we can accommodate continuous actions with the corresponding optimal rewards $\mathcal{U}^j(S)$ replaced by the function $\mathcal{U}(S, a)$ and a denoting action with continuous value.

As in the Markov optimal stopping problem, we can offer a numerical solution when the K transition densities $f^j(S_t | S_{t-1})$ are known and the state is scalar. Again we select a sufficiently large interval which we sample at the points $\{S_1, \dots, S_n\}$. This gives rise to the K vectors $\mathbf{U}^j = [\mathcal{U}^j(S_1), \dots, \mathcal{U}^j(S_n)]^\top$ which are the sampled form of the K solution functions. We also form the K matrices $\mathcal{F}^j$ using the corresponding transition densities as explained in Section III-C. Finally we consider the single reward vector $\mathbf{R} = [\mathcal{R}(S_1), \dots, \mathcal{R}(S_n)]^\top$. The system of equations (23) under a sampled form becomes

$$\mathbf{U}^j = \mathcal{F}^j \times \left\{ \mathbf{R} + \gamma \max_{1 \leq \ell \leq K} \mathbf{U}^\ell \right\}, \quad j = 1, \dots, K,$$

where the “max” is taken on an element-by-element basis over the K vectors $\mathbf{U}^\ell$. The solution of this system of equations can be obtained by applying the following iterative scheme

$$\mathbf{U}_t^j = \mathcal{F}^j \times \left\{ R + \gamma \max_{1 \leq \ell \leq K} \mathbf{U}_{t-1}^\ell \right\}, \quad \mathbf{U}_0^j = 0, \quad (25)$$

where $j = 1, \dots, K$, with the limits (as $t \rightarrow \infty$) of the K vector sequences $\{\mathbf{U}_t^j\}, j = 1, \dots, K$ approximating the sampled version of the optimal functions $\mathcal{U}^j(S)$. Let us now apply this methodology to a specific example.

Example with Two AR(1) Processes: We present a simple example with $K = 2$ actions. The corresponding Markov processes are both AR(1) and of the form 1) $S_t = 0.8S_{t-1} + 1 + \mathcal{W}_t$, 2) $S_t = 0.8S_{t-1} - 1 + \mathcal{W}_t$ with $\{\mathcal{W}_t\}$ i.i.d. standard Gaussians. We consider a reward function $\mathcal{R}(S)$ which is the same as the stopping cost of the previous example and depicted in Fig. 2(a) in gray. We also set the geometric discount factor equal to $\gamma = 0.8$.

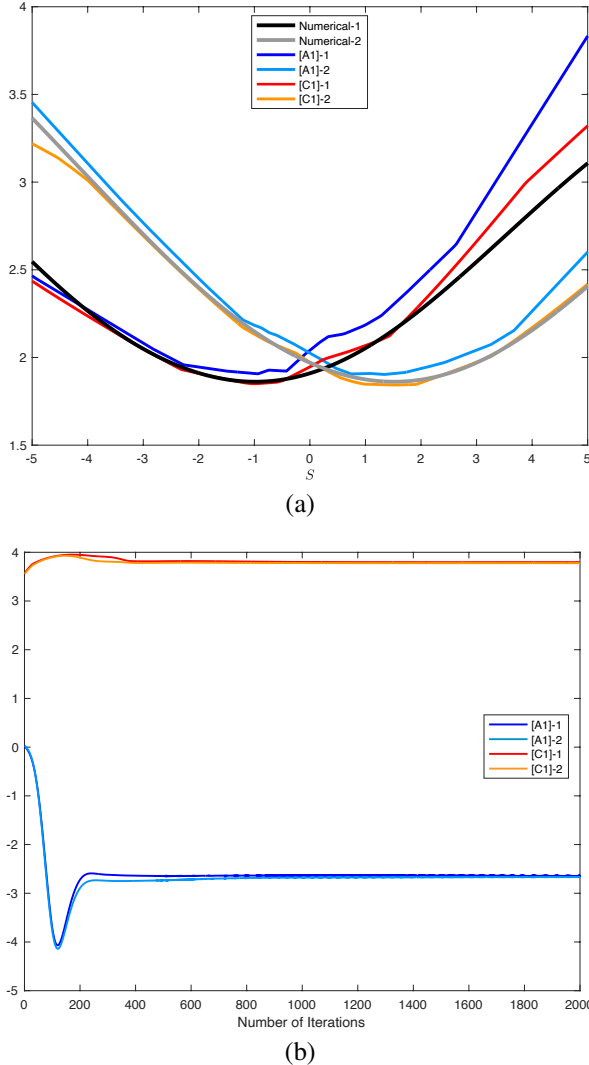


Fig. 3. (a) Numerical and Data-Driven estimation of the two expected optimal reward function $\mathcal{U}^1(S), \mathcal{U}^2(S)$ with $n = 500$ average number of samples per model, using two shallow networks with hidden layer of size 100 nodes. (b) Evolution of costs as a function of the number of iterations.

For the numerical solution we select the interval $[-20, 20]$ which we sample at 5000 points and apply the iteration presented in (25). The outcome can be seen in Fig. 3(a) with black and gray lines for $\mathcal{U}^1(S), \mathcal{U}^2(S)$ respectively. It is against these results that we need to compare the data-driven method.

For the data-driven estimation we randomly generate a length $n = 1000$ action sequence $\{a_t\}$ with $a_t \in \{1, 2\}$ which is used to generate 1000 realizations for the Markov controlled process. If $a_t = i$ then to go from S_t to S_{t+1} we use the i th Markov model. This means that on average we have 500 points per model. We apply [A1] and [C1] and for the latter, since $0.2 \leq \mathcal{R}(S) \leq 1$, we can show that $\frac{0.2}{1-\gamma} \leq \mathcal{U}^j(X) \leq \frac{1}{1-\gamma}$ suggesting that we can select the interval $\mathcal{I} = [1, 5]$ as our range. For both functions we select a shallow neural network with hidden layer of size 100 and ReLU activations. We run the algorithms for 2000 iterations using a step size $\mu = 0.001$ and forgetting factor for the power estimation $\lambda = 0.99$ of the ADAM version. Again knowledge of the range and using it in [C1] can produce better estimates. From Fig. 3(b) by following the evolution of the costs, we conclude that both iteration methods are stable and converge successfully.

Exploration vs Exploitation: Reinforcement Learning has become very popular due to the possibility of *Exploring* and *Exploiting*. Assuming that we start with sufficient number of initial data from each action, we can use them as described above to make an initial estimate of the functions $\mathcal{U}^j(S)$. Then these estimates can be used to decide about the next actions which constitutes the *Exploitation* phase.

If the initial data are not sufficient to guarantee an estimate of acceptable accuracy, or if the statistical behavior of the data changes with time then we must periodically make *random* decisions about the next action in order to activate possibilities that are not reachable by the existing action rule. These randomly generated data must be used to update the estimates of the functions $\mathcal{U}^j(S)$ and this is achieved mostly with the help of the SGD. This is known as the *Exploration* phase.

Whether at each step we should explore or exploit can be decided using randomization. Specifically, at each time t with probability $1 - \epsilon$ we could exploit and with probability ϵ explore. An asymptotic analysis could provide the limiting behavior of a scheme of this form in the stationary case when there is no change in the statistical behavior of the data but also in the case where the data are (slowly) varying in time and the updates attempt to *track* the change. Such an analysis is already available for adaptive algorithms for classical FIR filters [11], [12] and it would be extremely interesting if we were able to extend it to accommodate the neural network class.

REFERENCES

- [1] G.V. Moustakides, K. Basioti, “Training neural networks for likelihood/density ratio estimation,” *arXiv*: 1911.00405, 2019.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial networks,” *Comm. ACM*, vol. 63, no. 11, pp. 139–144, 2020.
- [3] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Math. Contr. Sig. Syst.*, vol. 2, no. 4, pp. 303–314, 1989.
- [4] K. Hornik, M. Tinchcombe, H. White, “Multilayer feedforward networks are universal approximators,” *Neural Net.*, vol. 2, pp. 359–366, 1989.

- [5] D.P. Kingma and J.L. Ba, “ADAM: A method for stochastic optimization,” *Proc. Intern. Conf. Learn. Repres.*, ICLR-2015
- [6] A.N. Shiryaev, *Optimal Stopping Rules*, Springer, 1978.
- [7] G. Peskir, A. Shiryaev, *Optimal Stopping and Free-Boundary Problems*, Birkhäuser, 2006.
- [8] A. Tartakovsky, I. Nikiforov and M. Basseville, *Sequential Analysis: Hypothesis Testing and Change-Point Detection*, Chapman & Hall, 2020.
- [9] R.S. Sutton and A.G. Burton, *Reinforcement Learning an Introduction*, MIT Press, Cambridge, Massachusetts, 4th printing, 2002.
- [10] A. Benveniste, M. Métivier and P. Priouret, *Adaptive Algorithms and Stochastic Approximations*, Springer, 1990.
- [11] E. Eleftheriou, D.D. Falconer, “Tracking properties and steady-state performance of RLS adaptive filter algorithms,” *IEEE Trans. ASSP*, vol. 34, no. 5, pp. 1097–1110, 1986.
- [12] B. Ninness, J.C. Gómez and S. Weller, “Frequency domain analysis of adaptive tracking algorithms,” *IFAC Proc.*, vol. 30, no. 11, pp. 1643–1648, 1997.